Terminal Basics

This fact sheet will help familiarize you with the terminal shell. In Linux and Mac, the application to access the shell is typically called "Terminal". In Windows, you can use Command Prompt (cmd.exe) or PowerShell (powershell.exe). If you have WSL installed on Windows, the `wsl` command can be run to open the WSL terminal.

Command Cheat Sheet

Here is a list of commonly used commands in the terminal shell. Note that these are for Mac/Linux, although PowerShell aliases many of its commands to match these. The commands for Command Prompt are totally different, in many cases.

- `man` (manual) - this command will print the help manual for whatever command/program you pass it, assuming that a manual is available for it. The manual can be navigated using `PgUp` and `PgDn`, and exited by pressing `q` on your keyboard. (On most modern Linux systems you can also scroll with the mouse)

- `pwd` (print working directory) - this command prints your current working directory in the file system. It is important to be aware of the directory you are in, and how to navigate the file system, so you are working in the correct location

- `ls` (list) - this command shows you the directory listing, i.e., the list of all the files in and subdirectories in the directory `ls` is querying. Below are the common options for `ls`:

    - `-l` - this switch changes the output to be in a more detailed list that shows you permissions, last access/modify date and time, and the size of file

    - `-a` - this switch shows hidden files and directories. In Linux, directories that start with a period are hidden
    - `-h` - this switch converts the bytes of a file's size into a more human-readable format, like kilobytes, megabytes, or gigabytes, depending on the size of the file
    - The switches can be combined like the following: `ls -lah` (the order of the switches is not important, `-ahl` would do the same thing)
    - By default, `ls` shows the listing for the current directory. But, you can pass it another directory as an argument: `ls -l /var/log`
- `cd` (change directory) - this command is used to change your working directory. It takes one argument, the directory you want to chage to.

- `mkdir` (make directory) - this command is used to create a new directory.

    - `-p` - this switch is used to create intermediate directories if they don't exist
- `cp` (copy) - this command is used to copy files or directories from a source to a destination. Below are some common switches:

    - `-b` - backup destination files
    - `-r` - recursively copy, this is used to copy directories and subdirectories. By default, directories are skipped without this flag
    - `-f` - force. If you try to copy over a file and that file is in use, it will attempt to force delete the destination file to complete the copy action
    - `-i` - interactive. This will prompt you to answer yes or no before overwriting a file. By default, existing files are overwritten without warning
    - `-n` - no-clobber. Blocks existing files from being overwritten

- `mv` (move) - this command will move a file from one location to another. Also, renaming a file is technically a "move" operation, so `mv` can be used to rename a file as well.
    - `-b` - backup destination files
    - `-f` - force. Always overwrite destination
    - `-i` - interactive. Prompt before overwrite
    - `-n` - no clobber. Never overwrite destination
    - `-u` - update. Only update destination if it is older than the source or does not exist

`'rm` (remove) - this command will delete files and directories. Note that this will delete files permanently (there is not a recycle bin when using `rm`) - `-f` - ignore if file is missing (in PowerShell, use `-Force` instead) - `-i` - prompt before each removal - `-r` - remove directories and their children recursively - `-d` - remove empty directories

Understanding the Command Line Environment

When using the terminal, you are interacting with your computer's file system using a text based interface. You are hopefully somewhat familiar with navigating your file system using your file explorer application provided by the operating system (File Explorer on Windows, Finder on Mac, Files on Ubuntu Linux, etc.). When you click your file explorer application, you get a GUI window that shows icons for the various files and folders you have. These files and folders themselves are contained within a folder in your file system. By default, Windows File Explorer opens up to a Favorites and Recently Accessed Files screen, which is not an actual folder. Rather, it is a collection of folders. On Mac, Finder opens up your home directory, which contains your Desktop, Documents, iTunes, etc. folders.
If you open a blank window in VS Code and open a terminal (using CTRL + ~), the terminal opens you your user directory (remember, directories in the terminal correspond to folders in file explorer). On Mac, your user

directory is likely `/Users/<your-username>`. On Windows, it is typically `C:\Users\<your-username>`. This folder contains your Desktop, Documents, Downloads, etc. folders. You can check this by running the `ls` command. This command *lists* the files and directories contained in the current directory. To check the name of the directory you are currently in, you can use the `pwd` (print working directory) command. So, what if I am in `/Users/john/`, and want to get to my `Desktop` folder? I can use the `cd` (change directory) command to do so. If I type `cd Desktop`, I will change directory to the Desktop folder within the current directory, `/Users/john`, that I am in. Now, if I run `pwd`, I will see that I am in `/Users/john/Desktop/`. If I run `ls`, I will see all the files and folders that are contained within my `Desktop` folder. You can verify this by checking your own Desktop folder versus the list of files you get running `ls`.
Your computer's directory has a tree-like structure. It starts with a root, which is represented by a `/` on Mac/Linux and `C:\` on Windows. Below is the typical, default directory structure of a Windows and Mac filesystem:
Note: your computer may vary slightly from this, that is okay

Windows

```
C:\
    Program Files
    Program Files (x86)
    Users
        Administrator
        Guest
        <your-username>
            Desktop
            Documents
                Photos
            Downloads
    Windows
```

Mac

```
/
    bin
    boot
    dev
    etc
    lib
    home
    media
    mnt
    proc
    opt
    proc
    run
    srv
    sys
    usr
    Users
        <your-username>
            Desktop
            Documents
            Downloads
            iTunes
    var
```

Linux and the Linux terminal for WSL and Chrome is the same as Mac, except there is no Users folder. Instead, user directories are in `/home`

For the sake of this class, you only need to worry about the folders and files that are the "branches" and "leaves" of directory that is your username, which is why I have only "expanded" those directories in the diagrams above.
Returning to the example from a couple paragraphs ago, I have changed directory into /Users/john/Desktop/. But, what if I want to go back a directory? Terminal shells reserve two periods, `..`, for this purpose. If I run the command `cd ..`, it will change my directory from /Users/john/Desktop/ to /Users/john/. Running `cd ..` again will change directory from /Users/john/ to /Users/. If I wanted to get back to my Desktop folder, I would first have to `cd john` to get back to /Users/john/, then `cd Desktop` to get to /Users/john/Desktop/. Alternatively, I could

provide the whole path to my Desktop folder, with the command `cd john/Desktop`. This would let me change from /Users/ to /Users/john/Desktop/ in a single command.
As an example, let's say that this is what my (simplified) directory structure looks like:

```
/
    Users
        john
            Desktop
                CSE233
                    lab1
                        main.cpp
                    lab2
                NET120
            Documents
            Downloads
```

Assume I am in a terminal, and run the command `pwd`. The output I get is `/Users/john/Downloads/`. I want to get to the main.cpp file in "/Users/john/Desktop/CSE233/lab1". If we look at the directory tree, we can see that both lab1 and lab2 are directories contained within the directory "CSE233". "CSE233" is contained in my "Desktop" directory, which is under my name.
Currently, I am in /Users/john/Downloads/. "Desktop" is directly below "john", so I first want to get to "/Users/john/". To do so, I can use `cd ..` to move up one directory. Running this command will change my location from `/Users/john/Downloads/` to `/Users/john/`. Now, I want to get to my "Desktop" folder. Since it is directly below "john" in the directory tree, I can use `cd Desktop` to get there. If I run `pwd`, my current directory is now `/Users/john/Desktop/`. However, lab1 and lab2 are in "CSE233", not Desktop. "CSE233" is directly below Desktop, so to get there I can use `cd CSE122`. I am now in `/Users/john/Desktop/CSE233`.
Now, I want to compile and run the `main.cpp` file in lab1. Therefore, I'll `cd lab1` to get into the `lab1` directory. To compile the program, I'll

enter `g++ main.cpp -o main` to create an executable called `main`. To run it, I'll type `./main`.
Below is an example of the terminal I/O for this example. Lines that start with `>` are user input lines. Lines that start with `#` are comments, and are not part of the actual terminal I/O. Lines that do not start with any special characters are the terminal output.

```
# checking my current directory
> pwd
/Users/john/Downloads
# changing directory to my /User/john home directory
> cd ..
# displaying file listing
> ls
Desktop
Documents
Downloads
# changing to Desktop and checking contents
> cd Desktop
> ls
CSE233
NET120
# changing to CSE233 and checking contents
> cd CSE233
> ls
lab1
lab2
# changing to lab1 and running project
> cd lab1
# checking the contents
> ls
main.cpp
# compiling main.cpp to main
> g++ main.cpp -o main
# running main
> ./main
Hello World!
# listing files in lab1
> ls
main
main.cpp
```